

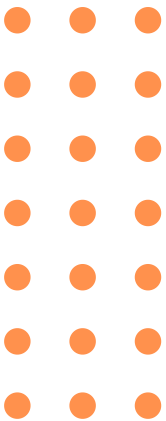
Investigacion 29/04/2025

Universidad Veracruzana



LENGUAJE DE MARCADO EXTENDIDO

XML



Realizado por:

Hernández Pérez Brandon

Núñez Sanchez Abraham



Indice

1. Introducción.....	3
2. Desarrollo.....	3
2.1. Documentos XML.....	3
a) Definición.....	3
b) Estructura de un documento XML (elementos, atributos, jerarquías).....	4
c) Reglas de formación (well-formed).....	5
d) Ejemplo de documento XML explicado.....	6
2.2. Esquemas XML.....	6
a) Definición y propósito de validación.....	6
b) Diferencias entre DTD y XML Schema (XSD).....	7
c) Ventajas de los Schemas frente a los DTD.....	9
d) Ejemplo de validación de un documento XML.....	9
e) Importancia en aplicaciones web.....	10
2.3. Tecnologías satélite de XML.....	11
a) XSLT (transformación de documentos XML).....	11
b) XPath (navegación en XML).....	12
c) Xquery.....	12
2.4. Aplicaciones en el desarrollo web.....	14
a) Uso de XML en sistemas web reales.....	14
b) Comparación breve con JSON.....	14
c) Ventajas y desventajas de XML.....	15
3. Bibliografía.....	16

1. Introducción

En el desarrollo de software y de aplicaciones web, el intercambio estructurado de información es un aspecto fundamental para garantizar la comunicación entre sistemas, plataformas y servicios. A lo largo del tiempo, han surgido diferentes tecnologías orientadas a resolver esta necesidad, entre las cuales destaca XML (eXtensible Markup Language), un lenguaje de marcado diseñado para almacenar, organizar y transportar datos de forma estructurada, legible y flexible.

XML fue desarrollado por el World Wide Web Consortium (W3C) como una solución estándar para representar información de manera independiente del sistema o lenguaje de programación utilizado. Su capacidad para definir estructuras personalizadas, validar documentos y facilitar la inter-operabilidad entre aplicaciones lo convirtió en una herramienta ampliamente utilizada en servicios web, configuración de software, intercambio de datos empresariales y múltiples tecnologías relacionadas.

El presente documento tiene como objetivo analizar los fundamentos de XML, su estructura interna y reglas de formación, así como estudiar el uso de esquemas de validación como DTD y XML Schema. Además, se abordan tecnologías complementarias como XSLT, XPath y XQuery, las cuales amplían las capacidades de procesamiento, transformación y consulta de documentos XML. Finalmente, se revisa la aplicación de XML en entornos web reales, sus ventajas, limitaciones y una breve comparación con formatos alternativos como JSON.

2. Desarrollo

2.1. Documentos XML

a) Definición

Es un metalenguaje que permite definir y almacenar datos, fue desarrollado por el World Wide Web Consortium (WC3) con el fin de almacenar datos en forma legible para cualquiera. Proviene del lenguaje SGML y permite definir la gramática de lenguajes específicos para estructurar documentos grandes. A diferencia de otros lenguajes, XML da soporte a bases de datos, siendo útil cuando varias aplicaciones deben comunicarse entre sí o integrar información. XML es una tecnología sencilla que tiene a su alrededor otras que la complementan y la hacen mucho más grande, con unas posibilidades mucho mayores. Tiene un papel muy importante en la actualidad ya que permite la compatibilidad entre sistemas para compartir la información de una manera segura, fiable y fácil.

El lenguaje de marcado extensible (XML) es un lenguaje de marcado que proporciona reglas para definir cualquier dato. A diferencia de otros lenguajes de programación, XML no puede realizar operaciones de computación por sí mismo. En cambio, se puede implementar cualquier software o lenguaje de programación para la administración estructurada de datos.

Por ejemplo, un documento de texto con comentarios. Los comentarios pueden ofrecer sugerencias como las siguientes:

- Ponga el título en negrita
- Esta oración es un encabezado
- Esta palabra es el autor

Estos comentarios mejoran la usabilidad del documento sin repercutir en su contenido. Del mismo modo, XML utiliza símbolos de marcado para proporcionar más información sobre los datos. Otros programas, como los navegadores y las aplicaciones de procesamiento de datos, utilizan esta información para procesar datos estructurados de manera más eficiente.

b) Estructura de un documento XML (elementos, atributos, jerarquías)

La tecnología XML busca dar solución al problema de expresar información estructurada de la manera más abstracta y re-utilizable posible. Que la información sea estructurada quiere decir que se compone de partes bien definidas, y que esas partes se componen a su vez de otras partes. Estas partes se llaman elementos, y se las señala mediante etiquetas. Una etiqueta consiste en una marca que señala una porción de este como un elemento. Un pedazo de información con un sentido claro y definido. Las etiquetas tienen la forma “<nombre>”, donde nombre es el nombre del elemento que se está señalando.

- Elemento: Los elementos XML pueden tener contenido (más elementos, caracteres o ambos), o bien ser elementos vacíos.
- Atributos: Los elementos pueden tener atributos, que son una manera de incorporar características o propiedades a los elementos de un documento. Deben ir entre comillas. Por ejemplo, un elemento «estudiante» puede tener un atributo «Mario» y un atributo «tipo», con valores «come croquetas» y «taleno» respectivamente.
- <Estudiante Mario="come croquetas" tipo="taleno">Esto es un día que Mario va paseando...</Estudiante>

c) Reglas de formación (well-formed)

Un documento XML se considera bien formado (well-formed) cuando cumple con las reglas sintácticas básicas definidas por el estándar XML, lo que permite que cualquier analizador sintáctico (parser) compatible pueda interpretarlo correctamente. Este concepto se diferencia de la validez, ya que un documento puede estar bien formado sin necesariamente cumplir un esquema o conjunto adicional de reglas definido mediante DTD o XML Schema.

Para que un documento XML sea considerado bien formado, debe cumplir con las siguientes reglas:

- Debe existir un único elemento raíz que contenga a todos los demás elementos del documento.
- Los elementos deben estar correctamente anidados, es decir, una etiqueta abierta debe cerrarse en el orden adecuado.

- Todas las etiquetas deben cerrarse correctamente, ya sea mediante una etiqueta de cierre o usando etiquetas vacías.
- Los valores de los atributos deben colocarse entre comillas simples o dobles.
- XML distingue entre mayúsculas y minúsculas, por lo que etiquetas como ``<nombre>`` y ``<Nombre>`` son consideradas diferentes.
- Los nombres de elementos y atributos deben seguir reglas de nomenclatura válidas y no contener caracteres inválidos.

Además, XML reconoce ciertos caracteres como espacios en blanco, tabulaciones y saltos de línea, los cuales pueden ser interpretados de manera específica por los procesadores. Las etiquetas, declaraciones y referencias de entidad forman parte del marcado XML, mientras que el contenido comprendido entre estas constituye la información o datos del documento.

El cumplimiento de estas reglas garantiza que la información pueda ser procesada de forma estructurada, consistente y compatible entre distintas aplicaciones.

d) Ejemplo de documento XML explicado

El lenguaje de marcado extensible (XML) es un lenguaje de marcado que proporciona reglas para definir cualquier dato. A diferencia de otros lenguajes de programación, XML no puede realizar operaciones de computación por sí mismo. En cambio, se puede implementar cualquier software o lenguaje de programación para la administración estructurada de datos.

Por ejemplo, imagine un documento de texto con comentarios. Los comentarios pueden ofrecer sugerencias como las siguientes:

- Ponga el título en negrita
- Esta oración es un encabezado
- Esta palabra es el autor

Estos comentarios mejoran la usabilidad del documento sin repercutir en su contenido. Del mismo modo, XML utiliza símbolos de marcado para proporcionar más información sobre los datos. Otros programas, como los navegadores y las aplicaciones de procesamiento de datos, utilizan esta información para procesar datos estructurados de manera más eficiente.

2.2. Esquemas XML

a) Definición y propósito de validación

Un esquema de lenguaje de marcado extensible, es un documento que describe algunas reglas o límites de la estructura de un archivo XML. Puede describir estas restricciones de varias maneras diferentes, como las siguientes:

- Reglas gramaticales para determinar el orden de los elementos
- Condiciones de Sí o No que el contenido debe cumplir
- Tipos de datos para el contenido de los archivos XML
- Restricciones de integridad de datos

Por ejemplo, un esquema XML para librerías podría imponer restricciones como las siguientes:

1. Un elemento de libro tendrá los atributos título y autor.
2. El elemento libro se anidará en un elemento de categoría con un nombre de atributo.
3. El precio de un libro será un elemento independiente anidado en libro.

Para cumplir con estas restricciones, se escribe el archivo XML como se muestra a continuación.

- `<nombre de la categoría="Tecnología">`
- `<título del libro="Learning Amazon Web Services", autor="Mark Wilkins">`
- `<precio>20 USD</precio>`
- `</libro>`
- `</categoría>`

Los esquemas XML refuerzan la coherencia en la forma en que las diferentes aplicaciones de software crean y usan los archivos XML. Algunas industrias implementan esquemas XML que son específicos de sus operaciones para reducir la complejidad de escribir código XML para la transferencia de datos entre empresas. Por ejemplo, los gráficos vectoriales escalables (SVG) son una especificación XML para describir datos relacionados con gráficos de

computadora. Los desarrolladores de software escriben archivos XML para que cumplan con las especificaciones de la industria.

b) Diferencias entre DTD y XML Schema (XSD)

- I. Document Type Definition (DTD): Restringir el tipo de información presente en el documento. Sin embargo, DTD no restringe en realidad los “tipos” en el sentido de tipos básicos como entero o cadena. En su lugar solamente restringe el aspecto de subelementos y atributos en un elemento. DTD es principalmente una lista de reglas que indican el patrón de sub-elementos que aparecen en un elemento.
 - A. Declaraciones tipo elemento: Los elementos deben ajustarse a un tipo de documento declarado en una DTD para que el documento sea considerado como válido.
 - B. Modelos de contenido: Un modelo de contenido es un patrón que establece los subelementos aceptados, y el orden en que se aceptan.
 - C. Declaraciones de lista de atributos: Los atributos se usan para añadir información adicional a los elementos de un documento.
 - D. Tipos de atributos
 1. Atributos CDATA y NMTOKEN
 2. Atributos enumerados y notaciones
 3. Atributos ID e IDREF
 - E. Declaración de entidades: XML hace referencia a objetos que no deben ser analizados sintácticamente según las reglas XML, mediante el uso de entidades. Las entidades pueden ser:
 1. Internas o externas
 2. Analizadas o no analizadas
 3. Generales o parametrizadas
 - F. Espacios de nombres: Los espacios de nombres XML permiten separar semánticamente los elementos que forman un documento XML.

II. XML Schema (XSD): Un Schema es algo similar a un DTD que superan muchas de las limitaciones y debilidades de las DTDs. Define qué elementos puede contener un documento XML, cómo están organizados y qué atributos y de qué tipo pueden tener sus elementos. Esta basado en la gramática y pensado para proporcionar una mayor potencia expresiva que las DTD que son menos capaces al describir los documentos a nivel formal. Los documentos esquema son más complejas, intentando superar sus puntos débiles y buscar nuevas capacidades a la hora de definir estructuras para documentos XML. El principal aporte de XML Schema es el gran número de tipos de datos que incorpora. XML Schema aumenta las posibilidades y funcionalidades de aplicaciones de procesamiento de datos, incluyendo tipos de datos complejos como fechas, números y strings.

A. Tipos de componentes: Fue diseñado completamente alrededor de namespaces y soporta tipos de datos típicos de los lenguajes de programación, como también tipos personalizados simples y complejos. Un esquema se define pensando en su uso final.

c) Ventajas de los Schemas frente a los DTD

- B. Usan sintaxis de XML, al contrario que los DTD.
- C. Permiten especificar los tipos de datos.
- D. Son extensibles.

d) Ejemplo de validación de un documento XML

La validación de un documento XML consiste en comprobar que su estructura y contenido cumplen con un conjunto de reglas previamente definidas en un esquema, ya sea mediante un DTD o un XML Schema (XSD). Esto permite asegurar que la información intercambiada entre sistemas sea consistente, correcta y utilizable.

Por ejemplo, supongamos un documento XML que almacena información de estudiantes:

```
<estudiantes>
```

```
<estudiante>
```

```
<nombre>Abraham</nombre>
```

```
<edad>20</edad>
```

```
<correo>abraham@email.com</correo>
```

```
</estudiante>
```

```
</estudiantes>
```

Para validar este documento, se puede definir un esquema XML (XSD) con reglas como:

- El elemento raíz debe ser <estudiantes>.
- Cada <estudiante> debe contener obligatoriamente nombre, edad y correo.
- El dato edad debe ser numérico.
- El correo debe seguir un formato de texto válido.

Un ejemplo simple de esquema XSD sería:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
```

```
<xs:element name="estudiantes">
```

```
<xs:complexType>
```

```
<xs:sequence>
```

```
<xs:element name="estudiante" maxOccurs="unbounded">
```

```
<xs:complexType>
```

```
<xs:sequence>
```

```
<xs:element name="nombre" type="xs:string"/>
```

```
<xs:element name="edad" type="xs:integer"/>
```

```
<xs:element name="correo" type="xs:string"/>
```

```
</xs:sequence>
```

```
</xs:complexType>
```

```
</xs:element>
```

```
</xs:sequence>
```

```
</xs:complexType>
```

```
</xs:element>
```

```
</xs:schema>
```

Si el documento XML cumple estas reglas, será considerado válido. En caso contrario, el procesador XML devolverá errores indicando qué elementos o datos incumplen el esquema definido.

e) Importancia en aplicaciones web

La validación de documentos XML tiene una gran importancia en aplicaciones web debido a que muchos sistemas dependen del intercambio estructurado de información entre clientes, servidores, APIs y servicios externos.

XML permite asegurar que los datos enviados y recibidos mantengan una estructura uniforme, evitando errores durante el procesamiento. En entornos web, XML ha sido ampliamente utilizado en tecnologías como SOAP, RSS, archivos de configuración y servicios empresariales. Gracias a los esquemas XML, es posible validar automáticamente documentos antes de ser procesados por una aplicación, reduciendo problemas relacionados con datos incompletos, formatos incorrectos o estructuras inválidas.

Además, XML facilita la interoperabilidad entre plataformas desarrolladas en distintos lenguajes de programación, ya que define un estándar común para representar información. Esto resulta especialmente útil en aplicaciones distribuidas, integración de sistemas y arquitecturas orientadas a servicios.

Aunque actualmente JSON es más popular en APIs modernas por ser más ligero y fácil de interpretar, XML continúa siendo relevante en sectores empresariales, financieros, gubernamentales y sistemas heredados donde la validación estricta, seguridad y robustez estructural son prioritarias.

2.3. Tecnologías satélite de XML

a) XSLT (transformación de documentos XML).

XSLT es un lenguaje declarativo que permite generar documentos a partir de documentos XML, XSLT es el documento que contiene el código fuente del programa, es decir, las reglas

de transformacion que se van a aplicar al documento inicial (el XML). XSLT se usa para obtener un documento diferente a partir de uno XML.

Al ser un lenguaje declarativo, las hojas de estilo no se escriben como una secuencia de instrucciones, sino como una coleccion de plantillas, cada plantilla establece como se transformar un determinado elemento (definido mediante Xpath).

Estas hojas siguen los siguientes pasos:

1. El procesador analiza el documento y construye el arbol del documento.
2. El procesador recorre el arbol del documento desde el nodo raiz.
3. En cada nodo recorrido, el procesador aplixa o no alguna plantilla.

Una hoja XSLT, contiene al menos las siguientes etiquetas:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0">
</xsl:stylesheet>
```

b) XPath (navegación en XML).

Xpath es un lenguaje que permite construir expresiones que recorren y procesan un documento XML. Similar a las expresiones regulares para seleccionar partes de un texto sin atributos (plain text), permite buscar y seleccionar teniendo en cuenta la estructura jerarquica de XML.

Xpath fue creado para se usado en el estandar XSLT y a si vez es la base sobre la que se han especificado nuevas herramientas para el tratamiento de documentos XML, es decir, sirve para decir como una hoja de estilo debe procesar el contenido de una pagina XML, pero tambien para poder poner enlaces o cargar en un navegador zonas determinadas de una pagina XML.

La forma en la que funciona y selecciona partes del documento XML es mediante la terminologia de: raiz, hijo, ancestro, descendiente. El tipo de expresion mas importante en Xpath es una ruta de ubicacion, una ruta de ubicacion consta de una secuencia de pasos de localizacion, por cada paso de localizacion se tienen 3 componentes.

- Un Eje (ancestor, attribute, descendant, clid,etc)
- Una Prueba de nodo (buscar todos los elementos llamados x cosa)
- 0 o mas Predicados (para filtrar datos de acuerdo a una condicion)

c) Xquery.

Lenguaje de consulta diseñada para colecciones de datos XML, semanticamente es similar a SQL, es un lenguaje de programacion funcional que consta en su totalidad de expresiones, no hay sentencias, para ejecutar una funcion, la expresion dentro del cuerpo de la misma se evalua y se retorna el resultado obtenido. La version 1.0 fue desarrollado por el grupo de trabajo de consulta XML del W3C.

Xquery proporciona los medios para extraer y manipular informacion de documentos XML o de cualquier fuente de datos que pueda ser representada mediante XML (como bases de datos relacionales o documentos ofimaticos).

Se usan expresiones de Xpath para acceder a determinadas partes del documento XML y añade expresiones similares a las de SQL (expresiones FLWOR = FOR, LET, WHERE, ORDER BY y RETURN). Este lenguaje se basa en el modelo en arbol de la informacion contenida en el documento XML que consiste en 7 tipos distintos de nodo los cuales son los siguientes:

- elementos
- atributos
- nodos de texto
- comentarios
- instrucciones de procesamiento
- espacios de nombres
- nodos de documentos

Un ejemplo de codigo Xquery es el siguiente. En este, se listan los personajes que aparecen en cada acto del Hamlet de Shakespeare, obtenidas en hamlet.xml

```
<html><head/><body>
```

```
{  
  for $act in doc("hamlet.xml")//ACT  
  let $speakers := distinct-values($act//SPEAKER)  
  return  
    <span>  
      <h1>{ $act/TITLE/text() }</h1>  
      <ul>  
        {  
          for $speaker in $speakers  
          return <li>{ $speaker }</li>  
        }  
      </ul>  
    </span>  
}  
</body></html>
```

2.4. Aplicaciones en el desarrollo web

a) Uso de XML en sistemas web reales.

XML es usado aun en diversos sistemas por su formato estandar para transportar datos entre aplicaciones, principalmente en arquitecturas basadas en servicios como SOAP, donde los mensajes viajan en formato XML. Mucho frameworks y servidores usan archivos XML para definir configuraciones, rutas, dependencias o comportamientos del sistema, facilitando la organizacion y permite que los sistemas sean mas faciles de mantener y escalar.

Algunos de los sistemas que siguen usando XML son los siguientes:

- SOAP: usa XML para estructurar mensajes entre servicios web. Permite comunicacion estandarizada y segura entre sistemas.
- Servicio web de Microsoft (.NET): usa XML en configuraciones y SOAP para integrar aplicaciones.
- Java EE: usado en archivos de configuracion para definir componentes y comportamiento de aplicaciones web.
- ATOM: usan XML para distribuir contenido actualizado, permitiendo que otros sistemas consuman esa informacion de manera automatica.

b) Comparación breve con JSON.

Aunque los dos son usados para el intercambio de datos entre aplicaciones, JSON es un formato abierto que puede ser leído tanto por personas como por máquinas y este es independiente de cualquier lenguaje de programación mientras que XML es un lenguaje de marcado que tiene reglas para definir cualquier tipo de dato y usa etiquetas para diferenciar entre los atributos de los datos y los datos reales. Con estas diferencias, JSON es la mejor opción.

Algunas similitudes y diferencias son las siguientes:

- Ambos son formatos de serialización de datos
- Permiten intercambiar datos entre diferentes aplicaciones
- XML usa una representación de árbol mientras que JSON usa pares clave-valor
- XML soporta mayor tipo de datos a diferencia de JSON, como los datos binarios y marcas de tiempo
- JSON es un poco más seguro que XML (XML es vulnerable a modificaciones no autorizadas, lo que crea un riesgo de seguridad (XXE))

c) Ventajas y desventajas de XML

Ventajas:

- Fácilmente procesable tanto por humanos como por software
- Separa la información de su presentación o formato

- Diseñado para ser usado en cualquier lenguaje o alfabeto
- Su analisis sintactico es facil debido a las reglas de un documento
- Estructura jerarquica
- Compatible con estandares web y otros formatos

Desventajas:

- Los archivos XML son mas grandes que json debido a la necesidad de etiquetas de apertura y cierre
- Su analisis y manipulacion suelen ser mas lentos y requieren mas recursos de CPU y memoria en comparacion con otros formatos.
- La creacion y mantenimiento es mas pesada que con otros formatos.
- Las bases de datos XML pueden tener limitaciones para definir permisos de acceso detallados
- Sensibles a ataques.

3. Bibliografía

- Amazon Web Services. (s.f.). *¿Cuál es la diferencia entre JSON y XML?*. Recuperado el 28 de abril de 2026, de <https://aws.amazon.com/es/compare/the-difference-between-json-xml/>
- Amazon Web Services. (s.f.). *¿Cuál es la diferencia entre JSON y XML?* Recuperado el 28 de abril de 2026, de <https://aws.amazon.com/es/compare/the-difference-between-json-xml/>
- Amazon Web Services, Inc. (s.f.). *¿Qué es XML? Explicación del lenguaje de marcado extensible (XML)*. <https://aws.amazon.com/es/what-is/xml/>
- Colaboradores de Wikipedia. (2024, 25 de enero). Definición de tipo de documento. Wikipedia, La enciclopedia libre. https://es.wikipedia.org/wiki/Definici%C3%B3n_de_tipo_de_documento
- Colaboradores de Wikipedia. (2026, 25 de enero). XML Schema. Wikipedia, La enciclopedia libre. https://es.wikipedia.org/wiki/XML_Schema
- Colaboradores de Wikipedia. (2026, 7 de abril). Extensible markup language. Wikipedia, La enciclopedia libre. https://es.wikipedia.org/wiki/Extensible_Markup_Language
- Just Sherekan. (2008, 16 de mayo). Introducción a XML. Just Sherekan - Blog de Programación. <https://web.archive.org/web/20080527235258/http://sherekan.com.ar/blog/2008/05/16/introduccion-a-xml/>
- MDN Web Docs. (2025, 20 de junio). Introducción a XML - XML: Lenguaje de marcado extensible. https://developer.mozilla.org/es/docs/Web/XML/Guides/XML_introduction
- Sintés Marco, B. (2025). XSLT. McLibre. <https://www.mclibre.org/consultar/xml/lecciones/xml-xslt.html>
- Wikipedia contributors. (s.f.). SOAP. En Wikipedia, The Free Encyclopedia. Recuperado el 28 de abril de 2026, de <https://es.wikipedia.org/wiki/SOAP>

- Wikipedia contributors. (s.f.). XPath: Predicados. En Wikipedia, The Free Encyclopedia. Recuperado el 28 de abril de 2026, de <https://es.wikipedia.org/wiki/XPath#Predicados>
- Wikipedia contributors. (s.f.). XQuery. En Wikipedia, The Free Encyclopedia. Recuperado el 28 de abril de 2026, de <https://es.wikipedia.org/wiki/XQuery>